

Data Mining Project Manual

Project Summary

This is one connected project. The full scenario is simple: you choose a YouTube video or channel, collect captions and comments, clean the text, discover repeated patterns, track how patterns change, group comments into themes, compare captions with comments, and then scale that work into a final project. The full workflow is designed as a pipeline, where the output of one stage becomes the input of the next stage.

The most important idea is that Lab 2 is the foundation for almost everything that comes after it. Once the comments and captions are cleaned into reusable files such as `cleaned_comments.csv` and `cleaned_captions.csv`, those outputs feed Labs 3, 4, 5, and 6. The extra project materials then show the final layer: students move from only running lab code to planning, evaluating, and presenting a full project in a structured way.

Full Project Flow At A Glance

Task	Stage	What happens here
1	Collection and exploration	Choose a suitable YouTube source and collect raw comments and captions.
2	Cleaning and preprocessing	Parse, normalize, tokenize, filter, and export clean reusable text files.
3	Pattern discovery	Find frequent words, co-occurrences, itemsets, and association rules.
4	Incremental and correlation analysis	Split comments into chunks and study how patterns change over time.
5	Clustering	Use TF-IDF and clustering to group comments into themes.
6	Synthesis and comparison	Compare captions and comments using keywords, n-grams, sentiment, and linked analyses.
7	Final project assembly	Scale up, plan scope, compare across videos/channels, and prepare the final project package.

Important Project Logic: If Task 2 Is Weak, Tasks 3–6 Become Unreliable. In Other Words, Bad Cleaning Leads To Bad Analysis.

Task 1: Collect and Explore Raw YouTube Data

Choose a suitable YouTube video or channel, collect raw captions and comments, and perform a first look at the dataset.

Main goal	Choose a suitable YouTube video or channel, collect raw captions and comments, and perform a first look at the dataset.
Main input(s)	A public YouTube video with English captions and enough real comments; raw files such as comments.txt and captions.vtt
Main output(s)	comments.txt; captions.vtt; basic EDA plots; early word-frequency summaries
Why this task matters	This task gives you the raw material for the whole project. It also helps you understand the difference between authored text and audience-generated text before you begin cleaning.

What you have to do

1. Select a neutral, educational, scientific, or general-knowledge video rather than a controversial or highly noisy source.
2. Check that English captions are available and that the video has enough real comments to analyze.
3. Download the captions, usually using yt-dlp with English auto-subtitles enabled.
4. Collect comments in a simple plain-text format. In this course, manual copy-paste is accepted because it is stable for beginners.
5. Load both raw files into Python and verify that the text can be read properly.
6. Run basic exploratory analysis such as caption length vs comment length, vocabulary diversity, and top word frequency after basic stopword removal.
7. Save the early plots and summary outputs so you can understand the nature of the source before cleaning it.

Recommended working order

- Choose source → collect captions → collect comments → load raw files → inspect structure → run first exploratory checks

Expected outputs

- comments.txt
- captions.vtt
- basic EDA plots
- early word-frequency summaries

Why this step is necessary

This task gives you the raw material for the whole project. It also helps you understand the difference between authored text and audience-generated text before you begin cleaning.

Practical notes for students

- Pick a source that is rich enough to analyze but not so chaotic that every comment is noise.
- Keep a backup of the raw files before editing anything.
- Treat this as data familiarization, not final analysis.

Task 2: Clean and Preprocess the Comments and Captions

Turn the raw files into clean, structured, reusable datasets that the later tasks can trust.

Main goal	Turn the raw files into clean, structured, reusable datasets that the later tasks can trust.
Main input(s)	Raw comments and captions from Task 1
Main output(s)	cleaned_comments.csv; cleaned_captions.csv; reusable cleaned_tokens columns
Why this task matters	This is the master key for the entire pipeline. The later pattern mining, temporal analysis, clustering, and synthesis stages all depend on the quality of this task.

What you have to do

1. Open the raw files and inspect their real format before writing any parser.
2. Parse comments into a structured table, usually with fields such as username, timestamp_text, and comment_text.
3. Parse captions into a structured form, usually sentence-level rows.
4. Normalize the text by lowercasing and removing URLs, punctuation, extra spaces, and other unwanted noise.
5. Tokenize the text into words.
6. Remove stopwords and very short tokens.
7. Apply lemmatization or stemming if appropriate for the chosen workflow.
8. Store the cleaned token lists in a column such as cleaned_tokens.
9. Drop empty rows created after cleaning.
10. Export cleaned_comments.csv and cleaned_captions.csv for later reuse.

Recommended working order

- Inspect file structure → parse comments → parse captions → normalize text → tokenize → remove stopwords → lemmatize/stem → export clean CSVs

Expected outputs

- cleaned_comments.csv
- cleaned_captions.csv
- reusable cleaned_tokens columns

Why this step is necessary

This is the master key for the entire pipeline. The later pattern mining, temporal analysis, clustering, and synthesis stages all depend on the quality of this task.

Practical notes for students

- Do not copy a parser blindly. Adapt it to your own file format.
- If you skip proper cleaning, the later results may look impressive but still be wrong.
- Keep the original columns if you may need timestamp or user information later.

Task 3: Discover Frequent Patterns and Word Relationships

Use the cleaned data to find repeated words, repeated word pairs, frequent itemsets, and association rules.

Main goal	Use the cleaned data to find repeated words, repeated word pairs, frequent itemsets, and association rules.
Main input(s)	cleaned_comments.csv and cleaned_captions.csv from Task 2
Main output(s)	co-occurrence tables; frequent itemsets; association rules; support-confidence plots; network or word-cloud visuals
Why this task matters	This task turns cleaned text into interpretable structure. Instead of only asking which words are common, you now ask which words repeatedly appear together and what that suggests about the discussion.

What you have to do

1. Load the cleaned datasets and verify that cleaned_tokens is usable.
2. Convert each row of cleaned tokens into a basket or transaction.
3. Remove baskets that are too short to be meaningful.
4. Count high-frequency unigrams so you know which words dominate.
5. Count co-occurring word pairs and keep only meaningful pairs that repeat often enough.
6. Visualize the strongest co-occurring pairs with charts or a network graph.
7. One-hot encode the transaction data and run Apriori to discover frequent itemsets.
8. Generate association rules and filter them using support, confidence, and lift.
9. Experiment with multiple support thresholds and compare what disappears or appears when the threshold changes.
10. Write short interpretations of the strongest patterns instead of only saving the output files.

Recommended working order

- Load clean tokens → build transactions → count unigrams/pairs → visualize co-occurrence → run Apriori → filter rules → compare settings → interpret insights

Expected outputs

- co-occurrence tables
- frequent itemsets
- association rules
- support-confidence plots
- network or word-cloud visuals

Why this step is necessary

This task turns cleaned text into interpretable structure. Instead of only asking which words are common, you now ask which words repeatedly appear together and what that suggests about the discussion.

Practical notes for students

- Frequency alone is not enough. A word can be common without being meaningful.
- Rule quality depends on filtering choices such as support and lift.
- Pattern discovery should end with interpretation, not only CSV files.

Task 4: Study How Patterns Change Over Time

Simulate incoming data by splitting the comments into chunks, then measure how important patterns rise, fall, or move together.

Main goal	Simulate incoming data by splitting the comments into chunks, then measure how important patterns rise, fall, or move together.
Main input(s)	cleaned_comments.csv from Task 2, plus the pattern-mining thinking from Task 3
Main output(s)	chunk-level unigram results; chunk-level pair results; trend plots; correlation matrix; short written explanations
Why this task matters	This task adds a dynamic view. Instead of treating the dataset as one frozen block, you learn to ask whether discussion themes stay stable or shift as more comments appear.

What you have to do

1. Load the cleaned comments file and confirm that cleaned_tokens exists.
2. Split the dataset into five chunks, usually by row order if true timestamps are not available.
3. Within each chunk, count the most frequent tokens and the most frequent co-occurring pairs.
4. Plot chunk-level frequency summaries so you can compare one segment with another.
5. Pick 5 to 10 patterns to track across all chunks.
6. Create frequency tables showing how each selected pattern changes from chunk 1 to chunk 5.
7. Compute correlations between the tracked patterns to see which ones rise or fall together.
8. Write short explanations for the most interesting movements or correlations.

Recommended working order

- Load clean comments → split into five chunks → mine patterns per chunk → track selected patterns → compute correlation → explain changes

Expected outputs

- chunk-level unigram results
- chunk-level pair results
- trend plots
- correlation matrix
- short written explanations

Why this step is necessary

This task adds a dynamic view. Instead of treating the dataset as one frozen block, you learn to ask whether discussion themes stay stable or shift as more comments appear.

Practical notes for students

- Row-based chunking is only a proxy for time, so mention that limitation in the report.
- Use the same cleaned dataset format as Task 2 to keep the pipeline consistent.
- The goal is not just plotting lines. The goal is explaining the change.

Task 5: Group Comments into Themes with Clustering

Convert comment text into vectors and use unsupervised learning to discover thematic groups.

Main goal	Convert comment text into vectors and use unsupervised learning to discover thematic groups.
Main input(s)	cleaned_comments.csv from Task 2
Main output(s)	TF-IDF matrix; elbow plot; K-Means clusters; optional DBSCAN comparison; PCA cluster visualization; cluster interpretations
Why this task matters	This task groups many individual comments into broader discussion themes. It is useful when you want to move from isolated comments to thematic audience segments or conversation types.

What you have to do

1. Load the cleaned comments file and validate its size and usable text fields.
2. Join cleaned token lists back into text strings so the comments can be vectorized.
3. Convert the comments into TF-IDF features.
4. Use the elbow method to estimate a suitable number of K-Means clusters.
5. Run K-Means and inspect the top representative terms for each cluster.
6. Read sample comments from each cluster so the interpretation stays connected to real text.
7. Optionally test DBSCAN as a second clustering method and compare its behavior with K-Means.
8. Visualize cluster positions using PCA.
9. Write a short interpretation of what each useful cluster appears to represent.

Recommended working order

- Prepare corpus → vectorize with TF-IDF → choose k → run clustering → inspect top terms and sample comments → visualize in PCA → interpret themes

Expected outputs

- TF-IDF matrix
- elbow plot
- K-Means clusters
- optional DBSCAN comparison
- PCA cluster visualization
- cluster interpretations

Why this step is necessary

This task groups many individual comments into broader discussion themes. It is useful when you want to move from isolated comments to thematic audience segments or conversation types.

Practical notes for students

- A cluster label by itself means nothing until you read its top terms and sample comments.
- Short YouTube comments often produce overlap, so not every cluster will be perfectly clean.
- K-Means may be easier to interpret on this type of data than DBSCAN, but that depends on the dataset.

Task 6: Synthesize the Whole Text-Mining Story

Compare captions and comments directly using keyword extraction, n-grams, sentiment, and linked analyses from the earlier tasks.

Main goal	Compare captions and comments directly using keyword extraction, n-grams, sentiment, and linked analyses from the earlier tasks.
Main input(s)	cleaned_comments.csv and cleaned_captions.csv from Task 2, plus outputs and ideas from Tasks 3–5
Main output(s)	TF-IDF keyword CSVs; bigram results; sentiment charts; overlap visuals; insight statements; a combined Lab 6 analysis folder
Why this task matters	This task is where the earlier methods become one story. You are no longer just mining patterns, clustering, or tracking changes separately. You are comparing the video's language with the audience's language and turning that into insight.

What you have to do

1. Load both cleaned datasets and join token lists back into text strings.
2. Run TF-IDF keyword extraction on comments and on captions separately.
3. Save the top keywords for both datasets.
4. Compare shared keywords and unique keywords between captions and comments.
5. Visualize the overlap, for example with a Venn diagram.
6. Run bigram analysis on both datasets and compare common two-word expressions.
7. Apply sentiment analysis to both comments and captions and compare their positive, neutral, and negative distributions.
8. Reuse Task 3 ideas by checking important co-occurring keyword pairs among the strongest terms.
9. Reuse Task 4 ideas by checking whether prominent keywords change over chunks when time-like structure exists.
10. Optionally reuse Task 5 ideas by clustering TF-IDF vectors to group thematic comment discussions.
11. Write 3 to 5 insight statements explaining how spoken content and audience reaction differ.

Recommended working order

- Run TF-IDF on both datasets → compare overlap and differences → analyze bigrams → compare sentiment → connect to past tasks → write insight statements

Expected outputs

- TF-IDF keyword CSVs
- bigram results
- sentiment charts
- overlap visuals
- insight statements
- a combined Lab 6 analysis folder

Why this step is necessary

This task is where the earlier methods become one story. You are no longer just mining patterns, clustering, or tracking changes separately. You are comparing the video's language with the audience's language and turning that into insight.

Practical notes for students

- This is the bridge to the final project. Save everything cleanly.
- The most useful outputs here are not only the plots; they are the final insight statements that explain them.
- At this point, your project has moved from method practice into genuine comparative analysis.

Task 7: Assemble the Full Final Project

So Task 7 has one job: take everything learned in Tasks 1–6 and turn it into a complete, well-scoped, well-explained project package.

Main goal	Scale the single-lab workflow into a finished comparative project.
Core inputs	Cleaned datasets, Lab 3 pattern results, Lab 4 temporal results, Lab 5 cluster themes, Lab 6 synthesis outputs.
Project toolkit support	Dashboard, Constraint Calculator, Execution Engine, and Resource Generator.
Minimum project shape	A clearly defined scope, multiple sources to compare, reusable analysis pipeline, and a final report.
Recommended scale	Move from one video toward three or more videos/channels where feasible.
Completion test	The project is complete when the comparison, interpretation, feasibility, and final packaging are all finished.

Task 7A: Choose and define the final project scope

- Decide whether the final project will compare multiple videos from one channel, multiple channels on the same topic, or a topic across a small set of related videos.
- Check feasibility before promising too much. The extra materials include a constraint calculator and a scope planner specifically to stop students from choosing a project that is too large for the available time.
- Write down the comparison question clearly. For example: How do captions and comments differ across several science videos, or how does audience sentiment change across multiple videos in one channel?

Task 7B: Build the scaled dataset

- Repeat the Task 1 and Task 2 pipeline for each chosen video or channel source so that all files follow the same structure.
- Keep file naming consistent so the analysis can be reused without confusion.
- Create a multi-video dataset or a folder structure that separates each source cleanly.

Task 7C: Reuse the core analyses

- Reuse Task 3 to compare co-occurrence, itemsets, and rule patterns across the selected sources.
- Reuse Task 4 to compare temporal or chunk-based movement across sources where appropriate.
- Reuse Task 5 to compare thematic clusters and audience discussion groups.
- Reuse Task 6 to compare keyword overlap, bigrams, sentiment, and audience-vs-caption differences across the broader dataset.

Task 7D: Use the Research Staircase / Toolkit structure

- Deconstruct: understand the topic, source, or target paper clearly before criticizing or extending it.
- Evaluate: identify strengths, weaknesses, hidden assumptions, and fair comparison points.
- Synthesize: generate extension ideas, check scope feasibility, and convert the project into a report plan.
- The same extra materials also describe specific support pieces:
- Dashboard: a browser-based overview that visualizes the workflow and the research staircase.
- Constraint Calculator / Scope Planner: a tool to check whether a project idea is realistic for the available time.
- Execution Engine: a prompt-based protocol for structured analysis using deconstruct → evaluate → synthesize logic.
- Resource Generator: a helper for finding better papers or datasets so students do not start with weak material.

Task 7E: Decide the final output structure

- Use a final report structure that first explains the original problem or source context, then presents the new project extension or comparison work.
- Present methods, implementation, results, discussion, and conclusion in a logical order.
- Do not dump plots without explanation. Every figure or table should support a concrete insight.

Task 7F: Final deliverables students should prepare

- A cleaned multi-source dataset or a clearly organized set of cleaned files.
- Saved outputs from the reused analyses: keyword files, bigram files, sentiment results, cluster results, pattern tables, and temporal comparisons as relevant.
- Visuals that support the comparison: charts, overlap diagrams, cluster plots, sentiment plots, and trend plots.
- A written set of insight statements that explain what changed, what stayed the same, and what the comparison means.
- A final report or manual that explains the problem, the method, the results, the interpretation, and the project scope decisions.

When the full project is complete

The full project is complete only when all seven tasks are finished together. That means the student has not only collected, cleaned, mined, tracked, clustered, and synthesized one dataset, but has also scaled that workflow into a final comparative project with a realistic scope and a finished report. At that point, the course goal is met: the student understands both the step-by-step process and the complete project scenario from beginning to end.